

Rapport – Devoir 3

AWS Free Data Lake

INF 438 – Bases de Données Avancées

1 Introduction

Ce projet consiste à concevoir et implémenter une architecture de lac de données (Data Lake) sur Amazon Web Services (AWS). L'objectif est de mettre en pratique les trois étapes fondamentales du traitement de données : la collecte et le stockage, la transformation ETL (Extract, Transform, Load), et l'analyse avec visualisation.

Les services AWS utilisés comprennent Amazon S3 pour le stockage distribué, AWS Lambda pour les transformations serverless, Amazon Athena pour les requêtes SQL, et Amazon CloudWatch pour la surveillance. Cette architecture permet de traiter efficacement des données de commandes e-commerce.

2 Étapes Réalisées

2.1 Partie 1 : Collecte et Stockage des Données

Un bucket S3 nommé `kaan-colakoglu-data-lake` a été créé dans la région `eu-central-1` (Francfort) avec deux dossiers principaux : `raw/` pour les données brutes et `processed/` pour les données transformées.

Le script Python `generate_data.py` a généré 500 commandes fictives avec des attributs variés : identifiants de commande et client, noms de produits (10 types), prix aléatoires, quantités, dates de commande distribuées sur 30 jours, villes turques (Istanbul, Ankara, Izmir, etc.), pays européens, et statuts de commande. Les données ont été exportées au format JSON Lines et téléchargées vers `s3://kaan-colakoglu-data-lake/raw/orders_data.json`.

Voir captures : images/bucket_structure.png, images/orders_data_json.png

2.2 Partie 2 : Transformation ETL avec Lambda

Un rôle IAM `LambdaS3ETLRole` a été configuré avec les permissions `AWSLambdaBasicExecutionRole` (pour CloudWatch) et `AmazonS3FullAccess`. La fonction Lambda `OrdersETLFunction` a été créée avec Python 3.12.

Le code de transformation effectue les opérations suivantes :

- Lecture des données depuis `raw/orders_data.json`
- Calcul du prix total : `totalPrice = price * quantity`
- Normalisation des données géographiques en majuscules
- Ajout d'un horodatage `processedAt`
- Génération de statistiques (nombre total de commandes, revenu total)
- Écriture de deux fichiers : `orders_processed.json` (avec statistiques) et `orders_lines.json` (format JSON Lines)

Les tests d'exécution ont confirmé le traitement réussi des 500 commandes avec logs visibles dans CloudWatch.

Voir captures : images/lambda_function.png, images/lambda_execution.png, images/orders_processed.png

2.3 Partie 3 : Interrogation avec Amazon Athena

Une table Athena a été créée dans la base de données `datalake` pointant vers le fichier `processed/orders_lines.json` au format JSON Lines. Cinq requêtes SQL ont été exécutées avec succès :

1. **Afficher toutes les commandes** : `SELECT * FROM datalake.orders_lines`
2. **Nombre total** : `SELECT COUNT(*)` – Résultat : 500 commandes
3. **Répartition par ville** : `GROUP BY city` – Identification des villes les plus actives
4. **Chiffre d'affaires par pays** : `SUM(totalprice) GROUP BY country`
5. **Top 5 produits (commandes)** : `COUNT(*) GROUP BY productname ORDER BY DESC LIMIT 5`

Voir captures : images/athena_table_creation.png, images/query[1-5]*.png

2.4 Partie 4 : Analyse et Visualisation

Le script `sales_analysis.py` a été développé pour effectuer une analyse complète :

Connexion et extraction : Utilisation de boto3 pour télécharger le fichier `processed/orders_lines.json` depuis S3, chargement en DataFrame pandas avec validation des types de données.

Analyses statistiques calculées :

- Résumé descriptif (moyenne, médiane, min, max, écart-type)
- Top 10 des villes par volume de ventes
- Top 5 des produits les plus commandés
- Distribution des statuts de commande (Completed, Pending, Shipped, Cancelled)
- Agrégation des ventes totales par pays

Dashboard de visualisation : Un graphique composite de 4 panneaux a été généré : graphiques en barres pour les villes et produits, graphique circulaire pour les statuts avec pourcentages, et graphique en barres pour les pays. Les données transformées par Lambda sont utilisées, affichant les villes et pays en majuscules (ISTANBUL, TURKEY).

Voir résultat : plots/sales_analysis.png

3 Problèmes Rencontrés

3.1 Erreur 404 HeadObject lors de l'accès à S3

Problème : An error occurred (404) when calling the HeadObject operation: Not Found

Cause : Le fichier cible n'existait pas encore dans le bucket S3 ou le chemin d'accès était incorrect.

Solution : Vérification minutieuse du nom du bucket et de la clé d'objet dans les variables d'environnement, puis upload manuel du fichier via la console S3.

3.2 Conflit de schéma Athena avec fichiers multiples

Problème : Lors de la création de la table Athena pointant vers le dossier `processed/`, deux fichiers avec des structures différentes étaient présents : `orders_processed.json` (structure imbriquée avec `statistics` et `orders`) et `orders_lines.json` (format JSON Lines avec un enregistrement par ligne). Athena ne pouvait pas déterminer un schéma unique cohérent pour la table.

Cause : Athena s'attend à ce que tous les fichiers d'un même dossier partagent le même schéma. La présence de deux formats différents crée une ambiguïté insurmontable pour l'inférence automatique du schéma.

Solution : Suppression du fichier `orders_processed.json` du dossier `processed/`, conservant uniquement `orders_lines.json` au format JSON Lines (un objet JSON par ligne). Ce format est directement interrogeable par Athena sans ambiguïté de schéma.

3.3 Limitations de S3 Select pour requêtes complexes

Contexte : L'énoncé du devoir proposait d'utiliser S3 Select pour interroger directement les fichiers JSON dans S3. Des requêtes simples ont été testées avec succès via S3 Select (lecture de toutes les commandes, comptage total), validant ainsi le fonctionnement de base de l'outil sur le format JSON Lines.

Limitation rencontrée : S3 Select présente des contraintes importantes pour les analyses complexes : il ne supporte pas les opérations d'agrégation avancées (GROUP BY, ORDER BY), ne permet pas de créer des tables avec schémas persistants, et nécessite de reconfigurer les paramètres pour chaque requête individuelle. Ces limitations rendent S3 Select inadapté pour les requêtes analytiques demandées (répartition par ville, chiffre d'affaires par pays, produits les plus vendus).

Solution adoptée : Utilisation d'Amazon Athena pour les requêtes d'agrégation. Athena offre une compatibilité SQL complète, permet de créer des tables réutilisables avec schémas définis, supporte toutes les opérations d'agrégation et de tri, et fournit une interface cohérente pour enchaîner plusieurs analyses.

4 Conclusion – Avantages de l’Architecture Data Lake

4.1 Avantages techniques

Scalabilité élastique : S3 peut gérer de quelques gigaoctets à plusieurs pétaoctets sans modification d’infrastructure. L’architecture s’adapte automatiquement à la croissance des données.

Modèle économique avantageux : Le modèle pay-as-you-go d’AWS permet de ne payer que pour les ressources réellement consommées, offrant une grande flexibilité financière.

Séparation des préoccupations : L’organisation en dossiers **raw/** et **processed/** respecte les bonnes pratiques : conservation des données originales intactes pour audit et retraitement, et séparation claire des étapes du pipeline.

4.2 Avantages analytiques

Requêtes SQL sans infrastructure : Athena permet d’interroger directement les fichiers S3 avec SQL standard, sans avoir besoin de charger les données dans une base de données relationnelle traditionnelle.

Architecture serverless moderne : Lambda élimine complètement la gestion de serveurs pour les transformations ETL. Pas de provisionnement, de maintenance ou de préoccupation de disponibilité.

Intégration aisée avec l’écosystème data : Les données dans S3 peuvent être facilement consommées par des outils d’analyse (pandas, matplotlib), des plateformes BI (QuickSight, Tableau), ou des frameworks Big Data (Spark, EMR).

Extensibilité architecturale : Cette base peut évoluer vers une architecture plus complexe en ajoutant AWS Glue pour le catalogage automatique, Amazon Redshift pour l’entreposage de données, ou Amazon QuickSight pour des tableaux de bord interactifs.

4.3 Apprentissages clés

Ce projet a permis de maîtriser les services cloud natifs AWS, de concevoir un pipeline ETL moderne et efficace, de comprendre l’importance du format et du schéma des données pour l’interrogation, et d’appliquer les bonnes pratiques de séparation des données brutes et traitées. L’architecture Data Lake s’avère particulièrement adaptée aux environnements où les données sont hétérogènes, volumineuses, et destinées à des analyses exploratoires.